

COMPLETE WEB DEVELOPMENT

Upper Secondary / Terminale Coursebook

HTML • CSS • JavaScript • Responsive Design • Projects

30 Complete Lessons • Practice Activities

Revision Tests • Answer Key • Reference Guides

A structured 80-page learning book for classroom teaching, independent study, portfolio building, and web-development exam preparation.

WEB DEVELOPMENT SERIES

GETTING STARTED

How to Use This Book

A practical roadmap for learning web development step by step.

What You Will Learn

Build semantic HTML pages, style them with modern CSS, add interactivity with JavaScript, work with APIs and storage, use Git, improve accessibility/performance, and deploy secure responsive projects.

Recommended Study Method

1. Read the lesson objectives. 2. Type every model code example yourself. 3. Change one value and observe the result. 4. Complete guided practice without the answers. 5. Build the independent mini-project. 6. Record errors and fixes in a debugging log.

Developer Editing Checklist

HTML: semantic structure and valid nesting. CSS: cascade, spacing, responsive layout. JavaScript: console errors, data types, event flow. Accessibility: labels, alt text, keyboard use, contrast. Performance: image size, requests, layout shift. Deployment: HTTPS, links, environment settings.

Suggested 6-Week Study Schedule

Week 1: Lessons 1-5 (web and HTML). Week 2: Lessons 6-10 (tables, forms, semantics, CSS). Week 3: Lessons 11-15 (typography, box model, layout, Flexbox, Grid). Week 4: Lessons 16-20 (responsive design and JavaScript basics). Week 5: Lessons 21-25 (functions, data, DOM, events, storage). Week 6: Lessons 26-30 + revision + final project.

REFERENCE

Table of Contents

Lessons 1-15

Lesson	Topic	Pages
1	Web and Internet Foundations	5-6
2	HTML Document Structure	7-8
3	Text, Headings, and Paragraphs	9-10
4	Links and Navigation	11-12
5	Images, Audio, and Video	13-14
6	Lists and Tables	15-16
7	Forms and Input	17-18
8	Semantic HTML and Accessibility	19-20
9	CSS Foundations	21-22
10	Selectors, Specificity, and Cascade	23-24
11	Colors, Units, and Typography	25-26
12	The CSS Box Model	27-28
13	Display and Position	29-30
14	Flexbox	31-32
15	CSS Grid	33-34

Part I Learning Map

Lessons 1-5: web foundations and core HTML. Lessons 6-10: lists, tables, forms, accessibility, CSS basics. Lessons 11-15: typography, box model, positioning, Flexbox, and Grid.

Part I Milestones

I can structure a semantic HTML page. I can create links, media, tables, and forms. I can apply maintainable CSS. I can build responsive one- and two-dimensional layouts.

REFERENCE

Table of Contents

Lessons 16-30, Revision and Reference

Lesson	Topic	Pages
16	Responsive Web Design	35-36
17	CSS Transitions and Animations	37-38
18	JavaScript Foundations	39-40
19	Variables, Data Types, and Operators	41-42
20	Conditions and Loops	43-44
21	Functions and Scope	45-46
22	Arrays and Objects	47-48
23	DOM Selection and Manipulation	49-50
24	Events and Form Validation	51-52
25	Web Storage and JSON	53-54
26	Fetch API and Asynchronous JavaScript	55-56
27	Git, GitHub, and Version Control	57-58
28	Web Performance and SEO	59-60
29	Security Basics and Deployment	61-62
30	Final Project and Debugging Review	63-64
Revision	Comprehensive Review Tests	65-72
Answers	Answer Key	73-78
Reference	HTML/CSS/JS Quick Reference	79
Reference	Web Development Quick Guide	80

Part II Learning Map

Lessons 16-20: responsive design and JavaScript foundations. Lessons 21-25: functions, data, DOM, events, and storage. Lessons 26-30: APIs, Git, performance, security, deployment, and final project.

Part II Milestones

I can create interactive browser features, work with JSON and APIs, use Git, diagnose problems in developer tools, improve accessibility/performance, and deploy a complete project.

LESSON 01 - CORE WEB DEVELOPMENT

Web and Internet Foundations

*Concepts, code, and model examples***Learning Objectives**

- Explain how the web, Internet, browsers, servers, and URLs work together.
- Distinguish a website, web page, domain name, hosting, and HTTP/HTTPS.
- Trace a simple request-response cycle.

Concept	Explanation	Example
Internet vs Web	The Internet is the global network infrastructure; the Web is a service that uses it.	A browser requests a page from a server.
Client and server	The client asks for resources; the server returns HTML, CSS, JavaScript, images, or data.	Browser -> HTTP request -> server -> response
URL	A URL identifies a resource and can include protocol, host, path, query, and fragment.	https://example.com/shop?id=5#top
HTTP/HTTPS	HTTP transfers web resources; HTTPS adds encryption and server authentication.	Use HTTPS for production sites.

Key Rules

A browser does not 'contain' the website; it downloads and renders resources. • DNS translates domain names into IP addresses. • A 404 status usually means the requested resource was not found.

Model Code

```
GET /index.html HTTP/1.1
Host: example.com

HTTP/1.1 200 OK
Content-Type: text/html
```

Study Tip

Type the model example for Web and Internet Foundations. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 01 - PRACTICE

Web and Internet Foundations

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a web and internet foundations feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What is the difference between the Internet and the Web?
2. Name the client in a normal web browsing session.
3. What does DNS do?
4. Identify the protocol in `https://example.com/about`.
5. What does HTTP status 404 normally mean?
6. Give one reason to prefer HTTPS.
7. Name two resources a server can return.
8. Write the main parts of a simple URL.

Mastery Challenge

Build one small example that demonstrates the main rule of Web and Internet Foundations. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. The Internet is the network; the Web is a service on it. 2. The web browser. 3. It maps domain names to IP addresses. 4. HTTPS. 5. Resource not found. 6. Encryption and authentication. 7. Examples: HTML, CSS, JavaScript, images, JSON. 8. Protocol, host/domain, path; optionally query and fragment.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use `console.log` for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 02 - CORE WEB DEVELOPMENT

HTML Document Structure

Concepts, code, and model examples

Learning Objectives

- Build a valid HTML5 document.
- Explain the roles of doctype, html, head, meta, title, and body.
- Use indentation to show nesting.

Concept	Explanation	Example
DOCTYPE	Declares HTML5 so browsers use standards mode.	<!doctype html>
head	Contains metadata, title, links to CSS, and other non-visible settings.	<head>...</head>
body	Contains content displayed in the page.	<body>...</body>
Nesting	Elements can contain other elements; closing order should match opening order.	Hello world

Key Rules

Use one html root element around head and body. • Set a character encoding such as UTF-8. • Give each page a meaningful title for usability and SEO.

Model Code

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello Web</h1>
```

Study Tip

Type the model example for HTML Document Structure. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 02 - PRACTICE

HTML Document Structure

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a html document structure feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What declaration starts a normal HTML5 document?
2. Which section contains visible page content?
3. Where does the title element belong?
4. Why use meta charset="utf-8"?
5. Correct the nesting: `<p><strong>Text</strong></p></strong>`.
6. What attribute can declare the document language?
7. Write an h1 inside body.
8. Why is indentation useful?

Mastery Challenge

Build one small example that demonstrates the main rule of HTML Document Structure. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. `<!doctype html>`. 2. body. 3. head. 4. To declare the character encoding. 5. `<p><strong>Text</strong></p>`. 6. lang, e.g. `<html lang="en">`. 7. `<h1>...</h1>`. 8. It makes nesting and structure easier to read and debug.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 03 - CORE WEB DEVELOPMENT

Text, Headings, and Paragraphs

Concepts, code, and model examples

Learning Objectives

- Structure text with headings and paragraphs.
- Use emphasis elements appropriately.
- Choose semantic elements instead of presentational hacks.

Concept	Explanation	Example
Headings	h1-h6 represent a hierarchy of section headings.	Main title
Paragraph	p groups a paragraph of prose.	Welcome to our site.
Emphasis	strong signals strong importance; em adds stress emphasis.	Warning now
Line break	br creates a line break inside content, not general spacing.	Address line 1 Address line 2

Key Rules

Use headings in logical order based on document structure. • Do not use repeated br elements to create layout spacing. • HTML describes meaning; CSS should control appearance.

Model Code

```
<article>
  <h1>Learning HTML</h1>
  <p>HTML gives content <strong>structure</strong>.</p>
  <h2>Why semantics matter</h2>
  <p>Meaning improves accessibility and maintenance.</p>
</article>
```

Study Tip

Type the model example for Text, Headings, and Paragraphs. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 03 - PRACTICE

Text, Headings, and Paragraphs

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a text, headings, and paragraphs feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What is the main purpose of h1-h6?
2. Which tag creates a paragraph?
3. Which element marks strong importance?
4. Which element adds stress emphasis?
5. Why should CSS handle visual spacing?
6. Is br appropriate for separating major sections?
7. Write a second-level heading.
8. Give one accessibility benefit of semantic headings.

Mastery Challenge

Build one small example that demonstrates the main rule of Text, Headings, and Paragraphs. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. To mark heading hierarchy. 2. p. 3. strong. 4. em. 5. Because HTML should describe structure and CSS presentation. 6. No; use structural elements and CSS. 7. <h2>...</h2>. 8. Screen-reader users can navigate a meaningful heading outline.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 04 - CORE WEB DEVELOPMENT

Links and Navigation

Concepts, code, and model examples

Learning Objectives

- Create internal and external hyperlinks.
- Use relative and absolute URLs.
- Build clear site navigation and meaningful link text.

Concept	Explanation	Example
Anchor	The a element creates a hyperlink using href.	About
Absolute URL	Includes protocol and full host.	https://example.com/docs
Relative URL	Points relative to the current document location.	../images/logo.png
Fragment	Links to an element id on the same or another page.	Contact

Key Rules

Use descriptive link text instead of vague 'click here' where possible. • target="_blank" opens a new tab; consider rel="noopener" for safety. • Navigation lists are commonly placed inside nav.

Model Code

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="#contact">Contact</a>
</nav>
<section id="contact">...</section>
```

Study Tip

Type the model example for Links and Navigation. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 04 - PRACTICE

Links and Navigation

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a links and navigation feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. Which attribute gives a link destination?
2. What is an absolute URL?
3. What is a relative URL?
4. How do you link to id="faq"?
5. Which semantic element commonly contains major navigation?
6. Why is 'Read the pricing guide' better than 'click here'?
7. What does target="_blank" do?
8. Write a link to about.html.

Mastery Challenge

Build one small example that demonstrates the main rule of Links and Navigation. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. href. 2. A complete URL including protocol and host. 3. A URL resolved from the current page location. 4. href="#faq". 5. nav. 6. It describes the destination and is clearer for accessibility. 7. Opens the link in a new browsing context, often a new tab. 8. About.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 05 - CORE WEB DEVELOPMENT

Images, Audio, and Video

*Concepts, code, and model examples***Learning Objectives**

- Embed responsive images and media.
- Write useful alternative text.
- Understand common media attributes and performance concerns.

Concept	Explanation	Example
Image	img embeds an image; src locates it and alt describes meaningful content.	<code></code>
Figure	figure and figcaption group media with a caption.	<code><figure>...<figcaption>...</figcaption></figure></code>
Audio	audio can provide built-in controls.	<code><audio controls src="intro.mp3"></audio></code>
Video	video can use controls, poster, and multiple source formats.	<code><video controls poster="cover.jpg">...</video></code>

Key Rules

Meaningful images need useful alt text; purely decorative images can use alt="". • Set width/height or CSS aspect-ratio where useful to reduce layout shift. • Compress media before publishing to improve page speed.

Model Code

```
<figure>
  
  <figcaption>Rainforest conservation area</figcaption>
</figure>
<video controls width="480">
  <source src="lesson.mp4" type="video/mp4">
</video>
```

Study Tip

Type the model example for Images, Audio, and Video. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 05 - PRACTICE

Images, Audio, and Video

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a images, audio, and video feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. Which attribute describes an image for users who cannot see it?
2. What alt value suits a purely decorative image?
3. Which element adds a caption to a figure?
4. How can you show built-in video controls?
5. Why compress large images?
6. What is a poster image on video?
7. Write an img element for logo.png with alt text 'GETI logo'.
8. Name one way to reduce layout shift from images.

Mastery Challenge

Build one small example that demonstrates the main rule of Images, Audio, and Video. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. alt. 2. alt="". 3. figcaption. 4. Use the controls attribute. 5. To reduce download size and improve performance. 6. An image shown before the video plays. 7. . 8. Provide dimensions or reserve space with CSS/aspect-ratio.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 06 - CORE WEB DEVELOPMENT

Lists and Tables

Concepts, code, and model examples

Learning Objectives

- Create ordered, unordered, and description lists.
- Build accessible data tables.
- Use table headers and captions correctly.

Concept	Explanation	Example
Unordered list	ul contains list items where order is not important.	HTML
Ordered list	ol is for sequence or ranking.	- Plan - Build
Table headers	th labels a row or column; scope can improve accessibility.	<th scope="col">Price</th>
Caption	caption gives the table a title or description.	<caption>Course fees</caption>

Key Rules

Use tables for tabular data, not page layout. • Keep table structure simple where possible. • Associate headers with data using semantic th elements and scope.

Model Code

```
<table>
  <caption>Course Fees</caption>
  <tr><th scope="col">Course</th><th scope="col">Fee</th></tr>
  <tr><td>HTML</td><td>$20</td></tr>
</table>
```

Study Tip

Type the model example for Lists and Tables. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 06 - PRACTICE

Lists and Tables

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a lists and tables feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

- Which element creates a bulleted list?
- Which element creates a numbered list?
- What is the purpose of li?
- Which element should label a table column?
- Why should tables not be used for page layout?
- What does caption do?
- Write a two-item ordered list.
- Give one accessibility advantage of th with scope.

Mastery Challenge

Build one small example that demonstrates the main rule of Lists and Tables. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. ul. 2. ol. 3. It represents a list item. 4. th. 5. Because tables are for data and layout tables harm semantics and accessibility. 6. It gives the table a title/description. 7. `<ol><li>One</li><li>Two</li></ol>`. 8. It helps assistive technology understand header-data relationships.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 07 - CORE WEB DEVELOPMENT

Forms and Input

Concepts, code, and model examples

Learning Objectives

- Build forms with labels and common controls.
- Use appropriate input types and attributes.
- Explain basic client-server form submission.

Concept	Explanation	Example
form	Groups controls and defines submission action/method.	<form action="/signup" method="post">
label	Connects descriptive text to a control using for/id.	<label for="email">Email</label>
input types	Specialized types improve mobile keyboards and validation.	email, number, date, password
button	Use button for explicit actions.	<button type="submit">Send</button>

Key Rules

Every important form control should have a clear label. • GET places data in the URL; POST sends data in the request body. • HTML validation helps users but does not replace server-side validation.

Model Code

```
<form action="/signup" method="post">
  <label for="email">Email</label>
  <input id="email" name="email" type="email" required>
  <button type="submit">Create account</button>
</form>
```

Study Tip

Type the model example for Forms and Input. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 07 - PRACTICE

Forms and Input

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a forms and input feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. Why connect label for to input id?
2. Which input type is suitable for email?
3. What attribute makes a field required?
4. Which method usually keeps submitted data out of the URL?
5. What does name on an input do?
6. Why is server-side validation still needed?
7. Write a submit button.
8. What is one benefit of type="date"?

Mastery Challenge

Build one small example that demonstrates the main rule of Forms and Input. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. It associates the label with the control for usability and accessibility.
2. email.
3. required.
4. POST.
5. It provides the key used when form data is submitted.
6. Client checks can be bypassed and the server must protect data integrity/security.
7. `<button type="submit">Submit</button>`.
8. Browsers can provide an appropriate date input UI and validation.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 08 - CORE WEB DEVELOPMENT

Semantic HTML and Accessibility

Concepts, code, and model examples

Learning Objectives

- Use semantic landmarks and meaningful elements.
- Apply accessible names, keyboard support, and logical structure.
- Recognize common accessibility mistakes.

Concept	Explanation	Example
Landmarks	header, nav, main, aside, and footer create meaningful page regions.	<main>...</main>
Semantic controls	Use native buttons and links rather than clickable divs for common actions.	<button>Save</button>
Accessible name	Visible labels, alt text, or ARIA labels identify controls and images.	<button aria-label="Close">X</button>
Keyboard	Interactive features should work without a mouse.	Tab to focus, Enter/Space to activate

Key Rules

Prefer native HTML semantics before adding ARIA. • Maintain sufficient color contrast and visible focus states. • Do not rely on color alone to communicate meaning.

Model Code

```
<header>...</header>
<nav aria-label="Primary">...</nav>
<main>
  <button type="button">Save changes</button>
</main>
<footer>...</footer>
```

Study Tip

Type the model example for Semantic HTML and Accessibility. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 08 - PRACTICE

Semantic HTML and Accessibility

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a semantic html and accessibility feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. Name three semantic landmark elements.

2. Why prefer a button over a clickable div for an action?

3. What should visible focus styles support?

4. Should color alone indicate an error?

5. What is ARIA used for?

6. What rule should guide ARIA use?

7. Give one way to provide an accessible name.

8. Why is keyboard testing important?

Mastery Challenge

Build one small example that demonstrates the main rule of Semantic HTML and Accessibility. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Examples: header, nav, main, aside, footer. 2. Button has built-in semantics, focus, and keyboard behavior. 3. Keyboard users must see which element is focused. 4. No; add text/icon/state information too. 5. To add accessibility semantics when native HTML is insufficient. 6. Use native HTML first; add ARIA only when necessary. 7. A visible label, alt text, or aria-label where appropriate. 8. Many users navigate without a mouse, and keyboard testing reveals interaction barriers.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 09 - CORE WEB DEVELOPMENT

CSS Foundations

Concepts, code, and model examples

Learning Objectives

- Connect CSS to HTML and write rules.
- Distinguish inline, internal, and external CSS.
- Understand selectors, declarations, properties, and values.

Concept	Explanation	Example
Rule	A CSS rule contains a selector and declaration block.	p { color: navy; }
External CSS	A linked stylesheet keeps styles reusable across pages.	<link rel="stylesheet" href="styles.css">
Property/value	Declarations set a property to a value.	font-size: 1rem;
Comments	CSS comments use /* ... */.	/* navigation styles */

Key Rules

Prefer external stylesheets for maintainable multi-page sites. • Use class selectors for reusable styling and ids mainly for unique identifiers/targets. • End each declaration with a semicolon for consistency.

Model Code

```
/* styles.css */
body {
  font-family: Arial, sans-serif;
  margin: 0;
}
h1 {
  color: #17324d;
}
```

Study Tip

Type the model example for CSS Foundations. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 09 - PRACTICE

CSS Foundations

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a CSS foundations feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What are the two main parts of a CSS rule?
2. How do you link styles.css from HTML?
3. What syntax starts a class selector?
4. What syntax starts an id selector?
5. Why prefer external CSS on larger sites?
6. How are CSS comments written?
7. Write a rule that makes p text 18px.
8. What does margin: 0 do?

Mastery Challenge

Build one small example that demonstrates the main rule of CSS Foundations. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Selector and declaration block. 2. <link rel="stylesheet" href="styles.css">. 3. A dot, e.g. .card. 4. A hash, e.g. #menu. 5. It improves reuse and maintainability. 6. /* comment */. 7. p { font-size: 18px; }. 8. It removes the element's default outer margin, subject to cascade/specificity.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 10 - CORE WEB DEVELOPMENT

Selectors, Specificity, and Cascade

Concepts, code, and model examples

Learning Objectives

- Use element, class, id, attribute, and combinator selectors.
- Explain cascade order and specificity.
- Avoid unnecessarily specific selectors.

Concept	Explanation	Example
Element selector	Matches elements by tag name.	p { ... }
Class selector	Matches elements with a class.	.card { ... }
Descendant selector	Matches elements inside another matching element.	nav a { ... }
Specificity	More specific selectors can override less specific ones when other cascade factors are equal.	#hero .title is more specific than .title

Key Rules

The cascade considers origin/importance, layers, specificity, and source order. • Avoid overusing !important because it makes overrides harder. • Keep selectors simple and component-oriented.

Model Code

```
.card p { color: #333; }
.card .price { font-weight: bold; }
#featured { border: 2px solid; }
input[type="email"] { width: 100%; }
```

Study Tip

Type the model example for Selectors, Specificity, and Cascade. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 10 - PRACTICE

Selectors, Specificity, and Cascade

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a selectors, specificity, and cascade feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

- Which selector matches `class="card"`?
- Which selector matches `id="hero"`?
- What does `nav a` select?
- Which is more specific: `p` or `.note`?
- When equal specificity rules conflict, what commonly breaks the tie?
- Why avoid frequent `!important`?
- Write an attribute selector for required inputs.
- What is the main purpose of specificity?

Mastery Challenge

Build one small example that demonstrates the main rule of Selectors, Specificity, and Cascade. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. `.card`. 2. `#hero`. 3. `a` elements that are descendants of `nav`. 4. `.note`. 5. Later source order, assuming other cascade factors are equal. 6. It makes styles harder to override and maintain. 7. `input[required]` { ... }. 8. It helps determine which competing declaration wins in the cascade.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use `console.log` for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 11 - CORE WEB DEVELOPMENT

Colors, Units, and Typography

Concepts, code, and model examples

Learning Objectives

- Use common color formats and CSS units.
- Build readable typographic systems.
- Choose relative units for flexible interfaces.

Concept	Explanation	Example
Colors	Common formats include hex, rgb/rgba, hsl/hsla, and named colors.	#0b7a75 / rgb(11 122 117)
Relative units	rem is based on the root font size; em depends on the current element context.	font-size: 1.25rem
Viewport units	vw and vh relate to viewport dimensions.	min-height: 100vh
Line height	Controls vertical spacing between lines of text.	line-height: 1.6

Key Rules

Use relative units for scalable text and spacing when practical. • Keep body text large enough to read comfortably. • Ensure foreground/background colors have sufficient contrast.

Model Code

```
html { font-size: 100%; }
body {
  font-family: system-ui, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
  color: #223;
}
h1 { font-size: clamp(2rem, 5vw, 4rem); }
```

Study Tip

Type the model example for Colors, Units, and Typography. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 11 - PRACTICE

Colors, Units, and Typography

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a colors, units, and typography feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What is a hex color example?
2. What is 1rem relative to?
3. What does vh measure?
4. Why use line-height around 1.5-1.7 for body text?
5. What does clamp() help with?
6. Why check color contrast?
7. Which is generally more scalable for text: rem or px?
8. Write a muted text color using hex.

Mastery Challenge

Build one small example that demonstrates the main rule of Colors, Units, and Typography. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Example: #17324d. 2. The root element font size. 3. A percentage of viewport height. 4. It improves readability by providing comfortable line spacing. 5. Responsive values within minimum and maximum limits. 6. For readability and accessibility. 7. rem. 8. Example: color: #667085;.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 12 - CORE WEB DEVELOPMENT

The CSS Box Model

Concepts, code, and model examples

Learning Objectives

- Explain content, padding, border, and margin.
- Use box-sizing to simplify sizing.
- Diagnose spacing and overflow problems.

Concept	Explanation	Example
Content box	The innermost area containing text or media.	width/height apply here by default
Padding	Space inside the border around content.	padding: 1rem
Border	Line surrounding padding/content.	border: 1px solid #ddd
Margin	Space outside the border.	margin: 0 auto

Key Rules

box-sizing: border-box makes declared width include padding and border. • Vertical margins can collapse in normal block layout. • Use browser developer tools to inspect computed box dimensions.

Model Code

```
*, ::before, ::after { box-sizing: border-box; }
.card {
  width: 320px;
  padding: 20px;
  border: 1px solid #ddd;
  margin: 24px auto;
}
```

Study Tip

Type the model example for The CSS Box Model. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 12 - PRACTICE

The CSS Box Model

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a the css box model feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. Name the four box model layers from inside outward.
2. What does padding affect?
3. What does margin affect?
4. Why use border-box?
5. What tool helps inspect the box model?
6. What can happen to adjacent vertical margins?
7. Write 16px padding on all sides.
8. If width is 300px with border-box, does padding increase the outer width beyond 300px?

Mastery Challenge

Build one small example that demonstrates the main rule of The CSS Box Model. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Content, padding, border, margin. 2. Space inside the border. 3. Space outside the border. 4. It makes width/height calculations more predictable. 5. Browser developer tools. 6. They can collapse in normal block flow. 7. padding: 16px;. 8. No, padding and border are included in the declared width.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 13 - CORE WEB DEVELOPMENT

Display and Position

Concepts, code, and model examples

Learning Objectives

- Use display values to control layout behavior.
- Explain normal flow and positioning schemes.
- Use z-index carefully with positioned elements and stacking contexts.

Concept	Explanation	Example
Block/inline	Block elements start on a new line; inline elements flow with text.	div vs span
inline-block	Flows inline but can accept width/height like a block.	display: inline-block
Position relative/absolute	Absolute elements are positioned relative to a containing block; relative can establish that context.	position: relative / absolute
Fixed/sticky	fixed attaches to viewport; sticky switches behavior based on scroll position.	position: sticky; top: 0

Key Rules

Use normal document flow before reaching for absolute positioning. • Absolute positioning removes the element from normal flow. • z-index only works meaningfully within stacking contexts and relevant positioning/layout contexts.

Model Code

```
.hero { position: relative; }
.badge {
  position: absolute;
  top: 1rem;
  right: 1rem;
}
.site-header { position: sticky; top: 0; }
```

Study Tip

Type the model example for Display and Position. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 13 - PRACTICE

Display and Position

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a display and position feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What is normal document flow?
2. Does position:absolute keep its original space in flow?
3. Which property offsets a sticky header from the top?
4. What can position:relative provide for an absolute child?
5. When is inline-block useful?
6. Why avoid absolute positioning for main page layout?
7. Write CSS to fix a help button 20px from bottom/right.
8. What property controls stacking order?

Mastery Challenge

Build one small example that demonstrates the main rule of Display and Position. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. The default layout process for document elements. 2. No. 3. top. 4. A positioning containing block/context. 5. When an inline-level box also needs dimensions/padding behavior like a block. 6. It is fragile for responsive layout and removes elements from normal flow. 7. position: fixed; bottom:20px; right:20px;. 8. z-index.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 14 - CORE WEB DEVELOPMENT

Flexbox

Concepts, code, and model examples

Learning Objectives

- Create one-dimensional flexible layouts.
- Align and distribute items on main and cross axes.
- Build common navigation and card layouts.

Concept	Explanation	Example
Flex container	display:flex activates flex formatting for direct children.	display: flex
Main axis	Controlled by flex-direction; justify-content distributes along it.	justify-content: space-between
Cross axis	align-items controls alignment across the main axis.	align-items: center
Flex growth	flex can control grow, shrink, and basis.	flex: 1 1 240px

Key Rules

Flexbox is ideal for one-dimensional rows or columns. • gap adds consistent spacing without margin hacks. • Use flex-wrap when items should move onto additional lines.

Model Code

```
.nav {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}
.cards { display: flex; flex-wrap: wrap; gap: 1rem; }
.card { flex: 1 1 240px; }
```

Study Tip

Type the model example for Flexbox. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 14 - PRACTICE

Flexbox

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a flexbox feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. How do you create a flex container?
2. Which property changes row vs column direction?
3. What does justify-content align along?
4. What does align-items align along?
5. What does gap do?
6. How can flex items wrap?
7. What is one good use case for flexbox?
8. Write CSS to center children horizontally and vertically in a flex container.

Mastery Challenge

Build one small example that demonstrates the main rule of Flexbox. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. display:flex. 2. flex-direction. 3. The main axis. 4. The cross axis. 5. Adds spacing between flex/grid items. 6. flex-wrap: wrap. 7. Examples: nav bars, toolbars, rows of cards, centered content. 8. display:flex; justify-content:center; align-items:center;.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 15 - CORE WEB DEVELOPMENT

CSS Grid

Concepts, code, and model examples

Learning Objectives

- Create two-dimensional page and component layouts.
- Define tracks with explicit and flexible sizing.
- Use auto-placement and responsive grid patterns.

Concept	Explanation	Example
Grid container	display:grid enables grid layout for direct children.	display: grid
Tracks	grid-template-columns/rows define columns and rows.	repeat(3, 1fr)
Fraction unit	fr divides available free space.	2fr 1fr
Responsive pattern	auto-fit/minmax create fluid columns.	repeat(auto-fit,minmax(220px,1fr))

Key Rules

Grid is powerful for two-dimensional layouts. • Use gap for row/column spacing. • Prefer content-driven responsive grids instead of many fixed breakpoints.

Model Code

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
.featured { grid-column: span 2; }
```

Study Tip

Type the model example for CSS Grid. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 15 - PRACTICE

CSS Grid

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a css grid feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. How do you start a grid container?
2. What does 1fr mean?
3. Which property defines columns?
4. What does gap do?
5. What does minmax(220px,1fr) mean?
6. When is Grid often preferable to Flexbox?
7. Write a 3-column equal grid.
8. What does grid-column: span 2 do?

Mastery Challenge

Build one small example that demonstrates the main rule of CSS Grid. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. display:grid. 2. One fraction of available grid free space. 3. grid-template-columns. 4. Adds spacing between grid tracks/items. 5. A track can be at least 220px and grow up to a fraction of available space. 6. For two-dimensional row-and-column layouts. 7. grid-template-columns: repeat(3, 1fr);. 8. Makes the item span two column tracks.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 16 - CORE WEB DEVELOPMENT

Responsive Web Design

Concepts, code, and model examples

Learning Objectives

- Build layouts that adapt to different screen sizes.
- Use fluid sizing, media queries, and responsive images.
- Apply a mobile-first approach.

Concept	Explanation	Example
Viewport meta	Tells mobile browsers to match layout viewport to device width.	<meta name="viewport" content="width=device-width, initial-scale=1">
Fluid layout	Uses percentages, fr, minmax, max-width, and flexible components.	max-width: 70rem
Media query	Applies styles when conditions are met.	@media (min-width: 768px) { ... }
Mobile first	Base styles target smaller screens; enhancements apply at larger widths.	min-width breakpoints

Key Rules

Choose breakpoints based on content needs, not device brands. • Avoid fixed widths that force horizontal scrolling. • Test touch targets, typography, and navigation on small screens.

Model Code

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
.layout { display: grid; gap: 1rem; }
@media (min-width: 768px) {
  .layout { grid-template-columns: 2fr 1fr; }
}
</style>
```

Study Tip

Type the model example for Responsive Web Design. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 16 - PRACTICE

Responsive Web Design

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a responsive web design feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. Why include the viewport meta tag?
2. What does mobile-first mean?
3. Which media feature commonly tests viewport width?
4. Should breakpoints follow phone model names?
5. Name one fluid sizing technique.
6. Why avoid fixed 1200px layouts on all screens?
7. Write a media query starting at 900px.
8. What should you test besides visual width?

Mastery Challenge

Build one small example that demonstrates the main rule of Responsive Web Design. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. To make mobile layout viewport and scaling behave as intended. 2. Write base styles for small screens, then enhance for larger screens. 3. min-width or max-width. 4. No; use content-driven breakpoints. 5. Percentages, fr, minmax, max-width, flexible images. 6. They can overflow narrow screens and harm usability. 7. @media (min-width: 900px) { ... }. 8. Touch targets, text readability, navigation, forms, performance, keyboard behavior.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 17 - CORE WEB DEVELOPMENT

CSS Transitions and Animations

Concepts, code, and model examples

Learning Objectives

- Add motion with transitions, transforms, and keyframes.
- Use motion to communicate state without harming accessibility.
- Respect reduced-motion preferences.

Concept	Explanation	Example
Transition	Animates a property change between states.	transition: transform .2s ease
Transform	Moves/scales/rotates without normal-flow re-layout.	transform: translateY(-2px)
Keyframes	Defines multi-step animations.	@keyframes pulse { ... }
Reduced motion	Users may request less animation.	@media (prefers-reduced-motion: reduce)

Key Rules

Animate transform and opacity when possible for smoother performance. • Do not make essential information depend on animation. • Provide reduced-motion alternatives for significant movement.

Model Code

```
.button { transition: transform .2s ease; }
.button:hover { transform: translateY(-2px); }
@media (prefers-reduced-motion: reduce) {
  * { scroll-behavior: auto; }
  .button { transition: none; }
}
```

Study Tip

Type the model example for CSS Transitions and Animations. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 17 - PRACTICE

CSS Transitions and Animations

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a css transitions and animations feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What does transition do?
2. Which two properties are often efficient to animate?
3. What does transform:scale(1.05) do?
4. What are @keyframes used for?
5. Why support prefers-reduced-motion?
6. Should animation be required to understand essential content?
7. Write a 0.3s opacity transition.
8. What does :hover represent?

Mastery Challenge

Build one small example that demonstrates the main rule of CSS Transitions and Animations. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. It animates a change between property values.
2. transform and opacity.
3. Scales the element to 105% of its original size.
4. To define multi-step animations.
5. Some users experience discomfort or need less motion.
6. No.
7. transition: opacity .3s ease;.
8. A pointer-hover interaction state.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 18 - CORE WEB DEVELOPMENT

JavaScript Foundations

Concepts, code, and model examples

Learning Objectives

- Explain JavaScript's role in web pages.
- Add scripts safely to HTML.
- Use comments, statements, and the browser console.

Concept	Explanation	Example
Role	JavaScript adds behavior, logic, data handling, and interaction.	menus, validation, dynamic content
Script loading	External scripts can be loaded with script src; defer waits for parsing before execution.	<script src="app.js" defer></script>
Console	console.log helps inspect values while debugging.	console.log('ready')
Comments	Use // for one line and /* ... */ for blocks.	// calculate total

Key Rules

Keep JavaScript in external files for maintainability on larger projects. • Use defer for scripts that depend on parsed HTML and do not need to block rendering. • Check the browser console for errors while developing.

Model Code

```
// app.js
console.log('Page ready');
const button = document.querySelector('#hello');
button.addEventListener('click', () => {
  alert('Hello!');
});
```

Study Tip

Type the model example for JavaScript Foundations. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 18 - PRACTICE

JavaScript Foundations

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a javascript foundations feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What is JavaScript mainly used for in the browser?

2. How do you load app.js externally?

3. What does defer help with?

4. Which method prints a value to the console?

5. How do single-line comments start?

6. Where can runtime errors usually be inspected?

7. What does alert() do?

8. Why keep scripts external?

Mastery Challenge

Build one small example that demonstrates the main rule of JavaScript Foundations. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Behavior, interaction, logic, and dynamic content. 2. <script src="app.js" defer></script>. 3. It allows HTML parsing to continue and runs the deferred script after parsing. 4. console.log(). 5. //. 6. Browser developer console. 7. Shows a browser dialog message. 8. Better organization, caching, and maintainability.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 19 - CORE WEB DEVELOPMENT

Variables, Data Types, and Operators

Concepts, code, and model examples

Learning Objectives

- Declare variables with let and const.
- Use common JavaScript data types.
- Apply arithmetic, comparison, and logical operators.

Concept	Explanation	Example
const	Declares a binding that cannot be reassigned.	const tax = 0.2;
let	Declares a block-scoped variable that can be reassigned.	let total = 0;
Primitive types	string, number, boolean, undefined, null, bigint, symbol.	'hello', 42, true
Operators	Perform arithmetic, comparison, assignment, and logical operations.	+, -, ===, &&,

Key Rules

Prefer const unless the variable needs reassignment. • Use === and !== for strict comparisons in most code. • Template literals use backticks and support \${expression}.

Model Code

```
const name = 'Mina';
let score = 0;
score += 10;
const passed = score >= 10;
console.log(`${name}: ${score}`);
```

Study Tip

Type the model example for Variables, Data Types, and Operators. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 19 - PRACTICE

Variables, Data Types, and Operators

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a variables, data types, and operators feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. When should you use `const`?
2. When should you use `let`?
3. Name three primitive data types.
4. What is the difference between `=` and `===`?
5. What does `&&` mean?
6. What does `||` mean?
7. Write a template literal showing name and age.
8. What does `score += 5` do?

Mastery Challenge

Build one small example that demonstrates the main rule of Variables, Data Types, and Operators. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. When the binding should not be reassigned. 2. When reassignment is needed. 3. Examples: string, number, boolean, undefined, null. 4. `=` assigns; `===` compares value and type strictly. 5. Logical AND. 6. Logical OR. 7. ``${name} is ${age}``. 8. Adds 5 to score and assigns the result back.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use `console.log` for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 20 - CORE WEB DEVELOPMENT

Conditions and Loops

Concepts, code, and model examples

Learning Objectives

- Control program flow with if/else and switch.
- Repeat work with for and while loops.
- Avoid common infinite-loop and logic errors.

Concept	Explanation	Example
if/else	Runs blocks according to a boolean condition.	if (score >= 50) { ... }
switch	Matches one expression against multiple case values.	switch(role) { case 'admin': ... }
for loop	Repeats with initialization, condition, and update.	for (let i=0; i<5; i++)
while loop	Repeats while a condition remains true.	while (items.length) { ... }

Key Rules

Make loop conditions eventually become false unless an intentional infinite loop is required. • Use break/continue carefully so control flow remains clear. • For arrays, for...of is often clearer than manual indexes when the index is not needed.

Model Code

```
const scores = [42, 78, 91];
for (const score of scores) {
  if (score >= 50) {
    console.log('Pass');
  } else {
    console.log('Retry');
  }
}
```

Study Tip

Type the model example for Conditions and Loops. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 20 - PRACTICE

Conditions and Loops

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a conditions and loops feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What does if do?
2. When can switch be useful?
3. What are the three common parts of a for loop?
4. What risk exists in while loops?
5. What does break do?
6. What does continue do?
7. Write a condition for age >= 18.
8. Which loop is convenient for iterating array values directly?

Mastery Challenge

Build one small example that demonstrates the main rule of Conditions and Loops. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Runs a block when a condition is true. 2. When selecting among multiple discrete cases. 3. Initialization, condition, update. 4. An infinite loop if the condition never becomes false. 5. Exits the nearest loop/switch. 6. Skips to the next loop iteration. 7. if (age >= 18) { ... }. 8. for...of.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 21 - CORE WEB DEVELOPMENT

Functions and Scope

Concepts, code, and model examples

Learning Objectives

- Write reusable functions.
- Pass parameters and return values.
- Explain block, function, and global scope.

Concept	Explanation	Example
Function declaration	Defines a reusable named function.	function add(a,b) { return a+b; }
Arrow function	Compact function expression syntax.	const add = (a,b) => a+b;
Parameters	Names that receive values passed to the function.	a and b
Return	Sends a result back to the caller and ends that function call.	return total;

Key Rules

Functions should do one clear job where practical. • Variables declared with let/const are block scoped. • Avoid unnecessary global variables because they increase coupling and collision risk.

Model Code

```
function calculateTotal(price, qty) {
  const subtotal = price * qty;
  return subtotal;
}
const total = calculateTotal(15, 3);
console.log(total);
```

Study Tip

Type the model example for Functions and Scope. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 21 - PRACTICE

Functions and Scope

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a functions and scope feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What is a function parameter?
2. What does return do?
3. What is an argument?
4. Where is a let variable declared inside a block available?
5. Why avoid too many globals?
6. Write a function greet(name) returning 'Hello ' + name.
7. What is an arrow function?
8. Can code after return in the same path run?

Mastery Challenge

Build one small example that demonstrates the main rule of Functions and Scope. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. A named input in a function definition. 2. Returns a value and ends the current function execution path. 3. A value passed when calling a function. 4. Inside that block and nested blocks, subject to normal scope rules. 5. They create hidden dependencies and naming conflicts. 6. `function greet(name) { return 'Hello ' + name; }`. 7. A concise function expression syntax using `=>`. 8. No, not after that return is executed.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use `console.log` for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 22 - CORE WEB DEVELOPMENT

Arrays and Objects

Concepts, code, and model examples

Learning Objectives

- Store collections in arrays and structured data in objects.
- Read and update values.
- Use common array methods.

Concept	Explanation	Example
Array	Ordered collection accessed by zero-based index.	<code>const colors = ['red','blue'];</code>
Object	Key-value structure representing a record/entity.	<code>const user = {name:'Ana', age:20};</code>
Array methods	<code>map</code> , <code>filter</code> , <code>find</code> , <code>reduce</code> , <code>forEach</code> support common operations.	<code>items.filter(x => x.active)</code>
Destructuring	Extracts values into variables.	<code>const {name} = user</code>

Key Rules

Array indexes start at 0. • Use dot or bracket notation to read object properties. • Prefer non-mutating patterns when state changes need to be predictable.

Model Code

```
const products = [
  { name: 'Book', price: 12 },
  { name: 'Pen', price: 2 }
];
const names = products.map(product => product.name);
const cheap = products.filter(product => product.price < 10);
```

Study Tip

Type the model example for Arrays and Objects. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 22 - PRACTICE

Arrays and Objects

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a arrays and objects feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What index is the first array item?
2. How do you access user.name?
3. What does map commonly return?
4. What does filter return?
5. What does find return?
6. Write an object with title and year.
7. What does products.length give?
8. What is destructuring used for?

Mastery Challenge

Build one small example that demonstrates the main rule of Arrays and Objects. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. 0. 2. user.name or user['name']. 3. A new array of transformed values. 4. A new array containing items that pass the test. 5. The first matching item, or undefined if none. 6. const item = { title: 'Web', year: 2026 };. 7. The number of items. 8. To extract array/object values into variables concisely.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 23 - CORE WEB DEVELOPMENT

DOM Selection and Manipulation

Concepts, code, and model examples

Learning Objectives

- Select elements from the Document Object Model.
- Change text, attributes, classes, and structure.
- Create and insert new elements safely.

Concept	Explanation	Example
querySelector	Returns the first element matching a CSS selector.	document.querySelector('.card')
querySelectorAll	Returns a static NodeList of all matches.	document.querySelectorAll('nav a')
textContent	Reads/sets text content without parsing HTML.	title.textContent = 'Welcome'
classList	Adds, removes, toggles, or checks classes.	menu.classList.toggle('open')

Key Rules

Prefer textContent for plain text to avoid accidental HTML injection. • Use createElement and append for structured dynamic content. • Cache DOM references when repeatedly used in the same logic.

Model Code

```
const title = document.querySelector('h1');
title.textContent = 'Dashboard';
const item = document.createElement('li');
item.textContent = 'New item';
document.querySelector('ul').append(item);
```

Study Tip

Type the model example for DOM Selection and Manipulation. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 23 - PRACTICE

DOM Selection and Manipulation

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a dom selection and manipulation feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What does `querySelector` return?
2. What does `querySelectorAll` return?
3. How do you change plain text safely?
4. How do you add class 'active'?
5. How do you create a div element?
6. Why can innerHTML be risky with untrusted input?
7. Write code to select `#menu`.
8. What method can append a created node?

Mastery Challenge

Build one small example that demonstrates the main rule of DOM Selection and Manipulation. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. The first matching element or null. 2. A `NodeList` of all matching elements. 3. Use `textContent`. 4. `element.classList.add('active')`. 5. `document.createElement('div')`. 6. It can create HTML injection/XSS if untrusted content is inserted. 7. `document.querySelector('#menu')`. 8. `append()` or `appendChild()`.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use `console.log` for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 24 - CORE WEB DEVELOPMENT

Events and Form Validation

Concepts, code, and model examples

Learning Objectives

- Respond to user actions with event listeners.
- Read form values and prevent default submission when needed.
- Implement basic client-side validation feedback.

Concept	Explanation	Example
Event listener	Runs a function when an event occurs.	button.addEventListener('click', handler)
Event object	Contains information about the event and target.	event.target
preventDefault	Stops the browser's default action for that event when cancelable.	event.preventDefault()
Validation	Checks user input before further processing.	if (!email.value.includes('@')) ...

Key Rules

Use input/change events for live form feedback appropriately. • Client-side validation improves UX but server-side validation remains essential. • Provide clear, accessible error messages near the affected field.

Model Code

```
const form = document.querySelector('form');
form.addEventListener('submit', event => {
  const email = document.querySelector('#email');
  if (!email.value) {
    event.preventDefault();
    email.focus();
  }
});
```

Study Tip

Type the model example for Events and Form Validation. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 24 - PRACTICE

Events and Form Validation

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a events and form validation feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What does `addEventListener` do?
2. Which event fires when a form is submitted?
3. What does `preventDefault` do?
4. Why still validate on the server?
5. How can you read an input's current value?
6. How can you focus an invalid field?
7. Write a click listener for `#save`.
8. What makes an error message accessible?

Mastery Challenge

Build one small example that demonstrates the main rule of Events and Form Validation. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Registers a handler for an event. 2. `submit`. 3. Stops the default browser action when possible. 4. Client-side checks can be bypassed. 5. `input.value`. 6. `input.focus()`. 7. `document.querySelector("#save").addEventListener("click", handler);`. 8. Clear text, programmatic association where needed, and not relying on color alone.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use `console.log` for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 25 - CORE WEB DEVELOPMENT

Web Storage and JSON

Concepts, code, and model examples

Learning Objectives

- Store small amounts of browser data using localStorage/sessionStorage.
- Serialize structured data with JSON.
- Handle missing or invalid stored data safely.

Concept	Explanation	Example
localStorage	Persists string data for an origin across browser sessions until cleared.	localStorage.setItem('theme','dark')
sessionStorage	Stores string data for the current tab/session.	sessionStorage.setItem('step','2')
JSON.stringify	Converts a JavaScript value to a JSON string.	JSON.stringify(settings)
JSON.parse	Converts a JSON string into a JavaScript value.	JSON.parse(text)

Key Rules

Storage APIs store strings, so objects usually need JSON serialization. • Do not store sensitive secrets in localStorage. • Use try/catch when parsing data that might be malformed.

Model Code

```
const prefs = { theme: 'dark', compact: true };
localStorage.setItem('prefs', JSON.stringify(prefs));
const raw = localStorage.getItem('prefs');
const saved = raw ? JSON.parse(raw) : {};
```

Study Tip

Type the model example for Web Storage and JSON. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 25 - PRACTICE

Web Storage and JSON

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a web storage and json feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. Does localStorage store objects directly?
2. What converts an object to JSON text?
3. What converts JSON text back to a value?
4. Which storage usually lasts beyond the tab session?
5. Why not store passwords/tokens casually in localStorage?
6. What does getItem return if a key is missing?
7. Write code to save language='en'.
8. Why might JSON.parse need try/catch?

Mastery Challenge

Build one small example that demonstrates the main rule of Web Storage and JSON. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. No; values are stored as strings. 2. JSON.stringify. 3. JSON.parse. 4. localStorage. 5. Scripts running on the origin may access it, so sensitive data can be exposed in attacks such as XSS. 6. null. 7. localStorage.setItem('language','en');. 8. Malformed JSON throws an exception.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 26 - CORE WEB DEVELOPMENT

Fetch API and Asynchronous JavaScript

Concepts, code, and model examples

Learning Objectives

- Request remote data with fetch.
- Use promises and async/await.
- Handle network and HTTP errors deliberately.

Concept	Explanation	Example
fetch	Starts an HTTP request and returns a Promise.	fetch('/api/courses')
Promise	Represents a future completion or failure of an async operation.	promise.then(...).catch(...)
async/await	Provides readable syntax for awaiting Promises inside async functions.	const res = await fetch(url)
JSON response	response.json() parses JSON body asynchronously.	const data = await res.json()

Key Rules

fetch rejects on many network failures but not automatically for every HTTP error status. • Check response.ok or status before using the body. • Wrap awaited operations in try/catch when failures must be handled.

Model Code

```
async function loadCourses() {
  try {
    const response = await fetch('/api/courses');
    if (!response.ok) throw new Error('Request failed');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

Study Tip

Type the model example for Fetch API and Asynchronous JavaScript. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 26 - PRACTICE

Fetch API and Asynchronous JavaScript

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a fetch api and asynchronous javascript feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What does fetch return?
2. What does await do inside an async function?
3. Why check response.ok?
4. How do you parse a JSON response body?
5. Where should recoverable errors be handled?
6. Does fetch automatically throw for every 404/500 response?
7. Write `fetch('/api/users')`.
8. What does catch receive?

Mastery Challenge

Build one small example that demonstrates the main rule of Fetch API and Asynchronous JavaScript. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. A Promise. 2. Pauses that async function until the Promise settles, then yields/throws accordingly. 3. Because HTTP errors can still resolve the fetch Promise. 4. `await response.json()`. 5. In catch or other explicit error handling. 6. No. 7. `fetch('/api/users')`. 8. The thrown/rejected error value.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use `console.log` for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 27 - CORE WEB DEVELOPMENT

Git, GitHub, and Version Control

Concepts, code, and model examples

Learning Objectives

- Explain repositories, commits, branches, and remotes.
- Use a basic Git workflow.
- Write meaningful commit messages and resolve simple collaboration issues.

Concept	Explanation	Example
Repository	A project directory tracked by Git, including history.	git init
Commit	A recorded snapshot with metadata/message.	git commit -m "Add contact form"
Branch	An independent line of development pointing to commits.	git switch -c feature/nav
Remote	A named reference to another repository, often on GitHub/GitLab.	git remote add origin ...

Key Rules

Commit small, coherent changes with descriptive messages. • Pull/fetch remote changes before pushing when collaborating. • Do not commit secrets, passwords, private keys, or environment files containing credentials.

Model Code

```
git init
git add .
git commit -m "Create landing page"
git branch -M main
git remote add origin <repository-url>
git push -u origin main
```

Study Tip

Type the model example for Git, GitHub, and Version Control. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 27 - PRACTICE

Git, GitHub, and Version Control

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a git, github, and version control feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What is a repository?
2. What is a commit?
3. What is a branch?
4. What does git add do?
5. What does git status show?
6. Why avoid committing secrets?
7. Write a command to create branch feature/forms.
8. What is a remote such as origin?

Mastery Challenge

Build one small example that demonstrates the main rule of Git, GitHub, and Version Control. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. A project tracked by Git with version history. 2. A recorded snapshot in the repository history. 3. A named line/pointer for independent development. 4. Stages changes for the next commit. 5. Working tree and staging status. 6. They can leak credentials and compromise systems. 7. `git switch -c feature/forms`. 8. A named reference to another repository location.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use `console.log` for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 28 - CORE WEB DEVELOPMENT

Web Performance and SEO

Concepts, code, and model examples

Learning Objectives

- Improve loading performance with practical front-end techniques.
- Use semantic metadata and content structure for search engines.
- Measure rather than guess performance problems.

Concept	Explanation	Example
Asset optimization	Compress images, minify bundles, and avoid unnecessary resources.	WebP/AVIF, code splitting
Caching	Lets browsers reuse previously downloaded resources.	Cache-Control headers
SEO basics	Use descriptive title, meta description, headings, meaningful content, and crawlable links.	<title>Web Development Course</title>
Measurement	Use browser performance tools and Lighthouse-like audits.	Largest Contentful Paint, layout shift

Key Rules

Optimize the largest and most important resources first. • Lazy-load below-the-fold images when appropriate. • SEO should not sacrifice accessibility, readability, or truthful content.

Model Code

```
<head>
  <title>Web Development Course | GETI</title>
  <meta name="description" content="Learn HTML, CSS, JavaScript, and responsive design.">
</head>

```

Study Tip

Type the model example for Web Performance and SEO. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 28 - PRACTICE

Web Performance and SEO

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a web performance and seo feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. Name one image optimization format.
2. What does browser caching help reduce?
3. Which HTML element gives the page title shown in browser/search contexts?
4. What is a meta description?
5. When is lazy loading useful?
6. Why measure performance?
7. Name one factor that can cause layout shift.
8. Should SEO use hidden keyword stuffing?

Mastery Challenge

Build one small example that demonstrates the main rule of Web Performance and SEO. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Examples: WebP or AVIF. 2. Repeated downloads and load time. 3. title. 4. A short summary of the page for metadata/search presentation. 5. For off-screen/below-the-fold resources that need not load immediately. 6. To identify actual bottlenecks and verify improvements. 7. Images/ads/components without reserved dimensions. 8. No; create useful, honest, accessible content.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 29 - CORE WEB DEVELOPMENT

Security Basics and Deployment

Concepts, code, and model examples

Learning Objectives

- Recognize common front-end security risks.
- Apply basic defensive practices.
- Explain static-site deployment, domains, HTTPS, and environment configuration.

Concept	Explanation	Example
XSS	Occurs when untrusted input becomes executable script/content in a page.	Avoid unsafe innerHTML with untrusted data
HTTPS	Encrypts traffic and authenticates the server certificate chain.	https://...
Secrets	API keys with real privileges should not be exposed in public client code.	Use server-side secrets/env variables
Deployment	Publishes built files to a host/CDN/server and connects domain/DNS.	GitHub Pages, Netlify, Cloudflare Pages, server hosting

Key Rules

Validate and encode data at the right boundaries. • Never trust client-side validation as a security boundary. • Keep dependencies updated and remove unused packages.

Model Code

```
// Safer plain-text rendering
output.textContent = userMessage;

// Avoid inserting untrusted data directly:
// output.innerHTML = userMessage;

// Production: use HTTPS and secure server-side validation.
```

Study Tip

Type the model example for Security Basics and Deployment. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 29 - PRACTICE

Security Basics and Deployment

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a security basics and deployment feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What is XSS?
2. Why can innerHTML be dangerous with untrusted data?
3. Should privileged secrets be embedded in public JavaScript?
4. What does HTTPS provide?
5. What is deployment?
6. Why keep dependencies updated?
7. What is one safer way to render plain user text?
8. Does client-side validation secure a server?

Mastery Challenge

Build one small example that demonstrates the main rule of Security Basics and Deployment. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Injection of script/content into pages so it executes in a user's context. 2. It can parse attacker-controlled HTML/script-like content. 3. No. 4. Encrypted transport and server authentication via certificates. 5. Publishing a site/app to hosting so users can access it. 6. Updates may fix security and stability issues. 7. Use `textContent`. 8. No; the server must validate and enforce security itself.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use `console.log` for JavaScript values. Change one thing at a time and retest before moving on.

LESSON 30 - CORE WEB DEVELOPMENT

Final Project and Debugging Review

Concepts, code, and model examples

Learning Objectives

- Plan and build a complete responsive website.
- Use a systematic debugging workflow.
- Review HTML, CSS, JavaScript, accessibility, performance, and deployment readiness.

Concept	Explanation	Example
Plan	Define users, goals, pages, content, components, and constraints before coding.	wireframe + content map
Build	Implement semantic HTML, responsive CSS, and focused JavaScript interactions.	component-by-component
Debug	Reproduce the problem, inspect errors/state, isolate the cause, fix, and retest.	console + devtools + small tests
Review	Check accessibility, mobile layouts, performance, links, forms, and deployment.	final checklist

Key Rules

Change one thing at a time when debugging difficult issues. • Read error messages carefully before rewriting large sections. • Test the final site on multiple viewport sizes and at least two major browsers when possible.

Model Code

PROJECT CHECKLIST

1. Semantic HTML and logical headings
2. Responsive layout with Flexbox/Grid
3. Accessible navigation and forms
4. JavaScript interaction with error handling
5. Optimized images and metadata
6. Git history and clean deployment
7. Final cross-device testing

Study Tip

Type the model example for Final Project and Debugging Review. Then change one meaningful value or selector and explain what changes in the browser.

Worked Analysis Routine

Step 1: Identify the target HTML/CSS/JavaScript feature. Step 2: Explain what it does. Step 3: Predict the browser result. Step 4: Test in developer tools. Step 5: Change one part and verify the new behavior.

LESSON 30 - PRACTICE

Final Project and Debugging Review

Common mistakes, guided practice, and mastery check

Common error	Better approach	Why?
Common mistake: using a final project and debugging review feature without checking structure, syntax, or browser behavior.	Better form: test the smallest valid example, then add complexity step by step.	Small verified steps reduce hidden syntax and logic errors.
Copying code without understanding it.	Type, run, modify, and explain the code.	Active testing builds transferable skills.

Guided Practice

1. What should happen before coding a large project?
2. Name three areas to test before deployment.
3. What is the first debugging step after noticing a bug?
4. Why isolate the smallest failing case?
5. Which tools reveal JavaScript errors and CSS layout details?
6. Why test keyboard navigation?
7. What should Git history contain?
8. Write one measurable final-project success criterion.

Mastery Challenge

Build one small example that demonstrates the main rule of Final Project and Debugging Review. Add a short code comment explaining the key idea, then test it in the browser and correct any errors.

Quick Answers

1. Plan users, goals, content, structure, and constraints. 2. Examples: accessibility, mobile layout, forms, links, performance, security, browsers. 3. Reproduce the bug reliably and observe the exact symptoms. 4. It makes the root cause easier to identify and reduces unrelated variables. 5. Browser developer tools: console, elements/inspector, network, performance panels. 6. To verify accessibility and interaction without a mouse. 7. Meaningful commits documenting coherent changes. 8. Example: all pages work at 320px-1440px without horizontal scrolling and core forms are keyboard usable.

Independent Practice

A. Rebuild the lesson example without copying. B. Add one improvement. C. Create one deliberate bug and debug it. D. Use the target feature in a mini real-life page such as a portfolio, school site, shop card, or dashboard.

Developer Strategy

Read the full error message. Inspect HTML structure and computed CSS. Use console.log for JavaScript values. Change one thing at a time and retest before moving on.

REVISION 01

Review Test 1 - Web & HTML Foundations

Mixed practice - complete without looking back at the lessons.

No.	Topic	Question
1	Web and Internet Foundations	What is the difference between the Internet and the Web?
2	Web and Internet Foundations	What does DNS do?
3	Web and Internet Foundations	What does HTTP status 404 normally mean?
4	Web and Internet Foundations	Name two resources a server can return.
5	HTML Document Structure	What declaration starts a normal HTML5 document?
6	HTML Document Structure	Where does the title element belong?
7	HTML Document Structure	Correct the nesting: <code><p>Text</p></code> .
8	HTML Document Structure	Write an h1 inside body.
9	Text, Headings, and Paragraphs	What is the main purpose of h1-h6?
10	Text, Headings, and Paragraphs	Which element marks strong importance?
11	Text, Headings, and Paragraphs	Why should CSS handle visual spacing?
12	Text, Headings, and Paragraphs	Write a second-level heading.

Writing / Project Task

Build or describe a mini page (80-120 words of explanation plus code or pseudocode) using at least four skills reviewed on this page. Label each target skill and explain one debugging decision.

Self-Assessment

10-12 correct: Strong mastery. 7-9: Good progress; review minor gaps. 4-6: Revisit related lessons and retype examples. 0-3: Study the core explanations before retrying.

REVISION 02

Review Test 2 - Media, Forms & Accessibility

Mixed practice - complete without looking back at the lessons.

No.	Topic	Question
1	Images, Audio, and Video	Which attribute describes an image for users who cannot see it?
2	Images, Audio, and Video	Which element adds a caption to a figure?
3	Images, Audio, and Video	Why compress large images?
4	Images, Audio, and Video	Write an img element for logo.png with alt text 'GETI logo'.
5	Lists and Tables	Which element creates a bulleted list?
6	Lists and Tables	What is the purpose of li?
7	Lists and Tables	Why should tables not be used for page layout?
8	Lists and Tables	Write a two-item ordered list.
9	Forms and Input	Why connect label for to input id?
10	Forms and Input	What attribute makes a field required?
11	Forms and Input	What does name on an input do?
12	Forms and Input	Write a submit button.

Writing / Project Task

Build or describe a mini page (80-120 words of explanation plus code or pseudocode) using at least four skills reviewed on this page. Label each target skill and explain one debugging decision.

Self-Assessment

10-12 correct: Strong mastery. 7-9: Good progress; review minor gaps. 4-6: Revisit related lessons and retype examples. 0-3: Study the core explanations before retrying.

REVISION 03

Review Test 3 - CSS Foundations

Mixed practice - complete without looking back at the lessons.

No.	Topic	Question
1	CSS Foundations	What are the two main parts of a CSS rule?
2	CSS Foundations	What syntax starts a class selector?
3	CSS Foundations	Why prefer external CSS on larger sites?
4	CSS Foundations	Write a rule that makes p text 18px.
5	Selectors, Specificity, and Cascade	Which selector matches class="card"?
6	Selectors, Specificity, and Cascade	What does nav a select?
7	Selectors, Specificity, and Cascade	When equal specificity rules conflict, what commonly breaks the tie?
8	Selectors, Specificity, and Cascade	Write an attribute selector for required inputs.
9	Colors, Units, and Typography	What is a hex color example?
10	Colors, Units, and Typography	What does vh measure?
11	Colors, Units, and Typography	What does clamp() help with?
12	Colors, Units, and Typography	Which is generally more scalable for text: rem or px?

Writing / Project Task

Build or describe a mini page (80-120 words of explanation plus code or pseudocode) using at least four skills reviewed on this page. Label each target skill and explain one debugging decision.

Self-Assessment

10-12 correct: Strong mastery. 7-9: Good progress; review minor gaps. 4-6: Revisit related lessons and retype examples. 0-3: Study the core explanations before retrying.

REVISION 04

Review Test 4 - Layout & Responsive Design

Mixed practice - complete without looking back at the lessons.

No.	Topic	Question
1	Display and Position	What is normal document flow?
2	Display and Position	Which property offsets a sticky header from the top?
3	Display and Position	When is inline-block useful?
4	Display and Position	Write CSS to fix a help button 20px from bottom/right.
5	Flexbox	How do you create a flex container?
6	Flexbox	What does justify-content align along?
7	Flexbox	What does gap do?
8	Flexbox	What is one good use case for flexbox?
9	CSS Grid	How do you start a grid container?
10	CSS Grid	Which property defines columns?
11	CSS Grid	What does minmax(220px,1fr) mean?
12	CSS Grid	Write a 3-column equal grid.

Writing / Project Task

Build or describe a mini page (80-120 words of explanation plus code or pseudocode) using at least four skills reviewed on this page. Label each target skill and explain one debugging decision.

Self-Assessment

10-12 correct: Strong mastery. 7-9: Good progress; review minor gaps. 4-6: Revisit related lessons and retype examples. 0-3: Study the core explanations before retrying.

REVISION 05

Review Test 5 - JavaScript Fundamentals

Mixed practice - complete without looking back at the lessons.

No.	Topic	Question
1	CSS Transitions and Animations	What does transition do?
2	CSS Transitions and Animations	What does transform:scale(1.05) do?
3	CSS Transitions and Animations	Why support prefers-reduced-motion?
4	CSS Transitions and Animations	Write a 0.3s opacity transition.
5	JavaScript Foundations	What is JavaScript mainly used for in the browser?
6	JavaScript Foundations	What does defer help with?
7	JavaScript Foundations	How do single-line comments start?
8	JavaScript Foundations	What does alert() do?
9	Variables, Data Types, and Operators	When should you use const?
10	Variables, Data Types, and Operators	Name three primitive data types.
11	Variables, Data Types, and Operators	What does && mean?
12	Variables, Data Types, and Operators	Write a template literal showing name and age.

Writing / Project Task

Build or describe a mini page (80-120 words of explanation plus code or pseudocode) using at least four skills reviewed on this page. Label each target skill and explain one debugging decision.

Self-Assessment

10-12 correct: Strong mastery. 7-9: Good progress; review minor gaps. 4-6: Revisit related lessons and retype examples. 0-3: Study the core explanations before retrying.

REVISION 06

Review Test 6 - Functions, Data & DOM

Mixed practice - complete without looking back at the lessons.

No.	Topic	Question
1	Functions and Scope	What is a function parameter?
2	Functions and Scope	What is an argument?
3	Functions and Scope	Why avoid too many globals?
4	Functions and Scope	What is an arrow function?
5	Arrays and Objects	What index is the first array item?
6	Arrays and Objects	What does map commonly return?
7	Arrays and Objects	What does find return?
8	Arrays and Objects	What does products.length give?
9	DOM Selection and Manipulation	What does querySelector return?
10	DOM Selection and Manipulation	How do you change plain text safely?
11	DOM Selection and Manipulation	How do you create a div element?
12	DOM Selection and Manipulation	Write code to select #menu.

Writing / Project Task

Build or describe a mini page (80-120 words of explanation plus code or pseudocode) using at least four skills reviewed on this page. Label each target skill and explain one debugging decision.

Self-Assessment

10-12 correct: Strong mastery. 7-9: Good progress; review minor gaps. 4-6: Revisit related lessons and retype examples. 0-3: Study the core explanations before retrying.

REVISION 07

Review Test 7 - Storage, APIs & Git

Mixed practice - complete without looking back at the lessons.

No.	Topic	Question
1	Web Storage and JSON	Does localStorage store objects directly?
2	Web Storage and JSON	What converts JSON text back to a value?
3	Web Storage and JSON	Why not store passwords/tokens casually in localStorage?
4	Web Storage and JSON	Write code to save language='en'.
5	Fetch API and Asynchronous JavaScript	What does fetch return?
6	Fetch API and Asynchronous JavaScript	Why check response.ok?
7	Fetch API and Asynchronous JavaScript	Where should recoverable errors be handled?
8	Fetch API and Asynchronous JavaScript	Write fetch('/api/users').
9	Git, GitHub, and Version Control	What is a repository?
10	Git, GitHub, and Version Control	What is a branch?
11	Git, GitHub, and Version Control	What does git status show?
12	Git, GitHub, and Version Control	Write a command to create branch feature/forms.

Writing / Project Task

Build or describe a mini page (80-120 words of explanation plus code or pseudocode) using at least four skills reviewed on this page. Label each target skill and explain one debugging decision.

Self-Assessment

10-12 correct: Strong mastery. 7-9: Good progress; review minor gaps. 4-6: Revisit related lessons and retype examples. 0-3: Study the core explanations before retrying.

REVISION 08

Review Test 8 - Performance, Security & Final Project

Mixed practice - complete without looking back at the lessons.

No.	Topic	Question
1	Web Performance and SEO	Name one image optimization format.
2	Web Performance and SEO	Which HTML element gives the page title shown in browser/search contexts?
3	Web Performance and SEO	When is lazy loading useful?
4	Web Performance and SEO	Name one factor that can cause layout shift.
5	Security Basics and Deployment	What is XSS?
6	Security Basics and Deployment	Should privileged secrets be embedded in public JavaScript?
7	Security Basics and Deployment	What is deployment?
8	Security Basics and Deployment	What is one safer way to render plain user text?
9	Final Project and Debugging Review	What should happen before coding a large project?
10	Final Project and Debugging Review	What is the first debugging step after noticing a bug?
11	Final Project and Debugging Review	Which tools reveal JavaScript errors and CSS layout details?
12	Final Project and Debugging Review	What should Git history contain?

Writing / Project Task

Build or describe a mini page (80-120 words of explanation plus code or pseudocode) using at least four skills reviewed on this page. Label each target skill and explain one debugging decision.

Self-Assessment

10-12 correct: Strong mastery. 7-9: Good progress; review minor gaps. 4-6: Revisit related lessons and retype examples. 0-3: Study the core explanations before retrying.

ANSWER KEY

Answer Key - Part 1

Use this section after completing the exercises independently.

Review Test 1 - Web & HTML Foundations

No.	Answer
1	The Internet is the network; the Web is a service on it.
2	It maps domain names to IP addresses.
3	Resource not found.
4	Examples: HTML, CSS, JavaScript, images, JSON.
5	<code><!doctype html></code> .
6	head.
7	<code><p>Text</p></code> .
8	<code><h1>...</h1></code> .
9	To mark heading hierarchy.
10	strong.
11	Because HTML should describe structure and CSS presentation.
12	<code><h2>...</h2></code> .

Review Test 2 - Media, Forms & Accessibility

No.	Answer
1	alt.
2	figcaption.
3	To reduce download size and improve performance.
4	<code></code> .
5	ul.
6	It represents a list item.
7	Because tables are for data and layout tables harm semantics and accessibility.
8	<code>OneTwo</code> .
9	It associates the label with the control for usability and accessibility.
10	required.
11	It provides the key used when form data is submitted.
12	<code><button type="submit">Submit</button></code> .

Correction Method

For every wrong answer, write the corrected code or statement and the rule that explains it. If the error repeats, return to the lesson and complete the Mastery Challenge again.

ANSWER KEY

Answer Key - Part 2

Use this section after completing the exercises independently.

Review Test 3 - CSS Foundations

No.	Answer
1	Selector and declaration block.
2	A dot, e.g. .card.
3	It improves reuse and maintainability.
4	p { font-size: 18px; }.
5	.card.
6	a elements that are descendants of nav.
7	Later source order, assuming other cascade factors are equal.
8	input[required] { ... }.
9	Example: #17324d.
10	A percentage of viewport height.
11	Responsive values within minimum and maximum limits.
12	rem.

Correction Method

For every wrong answer, write the corrected code or statement and the rule that explains it. If the error repeats, return to the lesson and complete the Mastery Challenge again.

Error Log

Record the item number, your answer, the correction, the browser/error message if any, and the final rule you learned.

ANSWER KEY

Answer Key - Part 3

Use this section after completing the exercises independently.

Review Test 4 - Layout & Responsive Design

No.	Answer
1	The default layout process for document elements.
2	top.
3	When an inline-level box also needs dimensions/padding behavior like a block.
4	position: fixed; bottom:20px; right:20px;.
5	display:flex.
6	The main axis.
7	Adds spacing between flex/grid items.
8	Examples: nav bars, toolbars, rows of cards, centered content.
9	display:grid.
10	grid-template-columns.
11	A track can be at least 220px and grow up to a fraction of available space.
12	grid-template-columns: repeat(3, 1fr);.

Correction Method

For every wrong answer, write the corrected code or statement and the rule that explains it. If the error repeats, return to the lesson and complete the Mastery Challenge again.

Error Log

Record the item number, your answer, the correction, the browser/error message if any, and the final rule you learned.

ANSWER KEY

Answer Key - Part 4

Use this section after completing the exercises independently.

Review Test 5 - JavaScript Fundamentals

No.	Answer
1	It animates a change between property values.
2	Scales the element to 105% of its original size.
3	Some users experience discomfort or need less motion.
4	<code>transition: opacity .3s ease;</code>
5	Behavior, interaction, logic, and dynamic content.
6	It allows HTML parsing to continue and runs the deferred script after parsing.
7	<code>//</code> .
8	Shows a browser dialog message.
9	When the binding should not be reassigned.
10	Examples: string, number, boolean, undefined, null.
11	Logical AND.
12	<code>`\${name} is \${age}`</code> .

Correction Method

For every wrong answer, write the corrected code or statement and the rule that explains it. If the error repeats, return to the lesson and complete the Mastery Challenge again.

Error Log

Record the item number, your answer, the correction, the browser/error message if any, and the final rule you learned.

ANSWER KEY

Answer Key - Part 5

Use this section after completing the exercises independently.

Review Test 6 - Functions, Data & DOM

No.	Answer
1	A named input in a function definition.
2	A value passed when calling a function.
3	They create hidden dependencies and naming conflicts.
4	A concise function expression syntax using <code>=></code> .
5	0.
6	A new array of transformed values.
7	The first matching item, or undefined if none.
8	The number of items.
9	The first matching element or null.
10	Use <code>textContent</code> .
11	<code>document.createElement('div')</code> .
12	<code>document.querySelector('#menu')</code> .

Correction Method

For every wrong answer, write the corrected code or statement and the rule that explains it. If the error repeats, return to the lesson and complete the Mastery Challenge again.

Error Log

Record the item number, your answer, the correction, the browser/error message if any, and the final rule you learned.

ANSWER KEY

Answer Key - Part 6

Use this section after completing the exercises independently.

Review Test 7 - Storage, APIs & Git

No.	Answer
1	No; values are stored as strings.
2	JSON.parse.
3	Scripts running on the origin may access it, so sensitive data can be exposed in attacks such as XSS.
4	localStorage.setItem('language','en');
5	A Promise.
6	Because HTTP errors can still resolve the fetch Promise.
7	In catch or other explicit error handling.
8	fetch('/api/users').
9	A project tracked by Git with version history.
10	A named line/pointer for independent development.
11	Working tree and staging status.
12	git switch -c feature/forms.

Review Test 8 - Performance, Security & Final Project

No.	Answer
1	Examples: WebP or AVIF.
2	title.
3	For off-screen/below-the-fold resources that need not load immediately.
4	Images/ads/components without reserved dimensions.
5	Injection of script/content into pages so it executes in a user's context.
6	No.
7	Publishing a site/app to hosting so users can access it.
8	Use textContent.
9	Plan users, goals, content, structure, and constraints.
10	Reproduce the bug reliably and observe the exact symptoms.
11	Browser developer tools: console, elements/inspector, network, performance panels.
12	Meaningful commits documenting coherent changes.

Correction Method

For every wrong answer, write the corrected code or statement and the rule that explains it. If the error repeats, return to the lesson and complete the Mastery Challenge again.

REFERENCE

HTML / CSS / JavaScript Quick Reference

Fast syntax guide for study and coding practice.

Feature	Syntax / Example
HTML document	<code><!doctype html> ... <head>...</head> <body>...</body></code>
Heading / paragraph	<code><h1>Title</h1> <p>Text</p></code>
Link / image	<code>Link </code>
Form	<code><label for="email">Email</label> <input id="email" name="email"></code>
CSS selector	<code>.card { padding: 1rem; }</code>
Flexbox	<code>display:flex; align-items:center; gap:1rem;</code>
Grid	<code>display:grid; grid-template-columns:repeat(3,1fr);</code>
Media query	<code>@media (min-width:768px) { ... }</code>
JS variable	<code>const name = "Ana"; let count = 0;</code>
Condition	<code>if (score >= 50) { ... } else { ... }</code>
Function	<code>function add(a,b) { return a+b; }</code>
DOM	<code>document.querySelector("#menu")</code>
Event	<code>button.addEventListener("click", handler)</code>
JSON	<code>JSON.stringify(obj) / JSON.parse(text)</code>
Fetch	<code>const res = await fetch(url); const data = await res.json();</code>
Git	<code>git add . -> git commit -m "message" -> git push</code>

Debugging Shortcuts

HTML/CSS: inspect elements and computed styles. JavaScript: read console errors, log values, use breakpoints. Network: inspect status codes and response payloads. Performance: check oversized images, blocking resources, and layout shift.

REFERENCE

Web Development Quick Guide

Final project, revision, and publishing checklist.

1. Structure

Use semantic HTML, one clear h1, logical headings, descriptive links, labeled forms, meaningful alt text, and accessible landmarks.

2. Style

Use external CSS, predictable box sizing, reusable classes, flexible spacing, readable typography, Flexbox/Grid, and mobile-first responsive rules.

3. Interactivity

Use small functions, const/let correctly, event listeners, safe DOM updates, validation feedback, error handling, and async requests with response checks.

4. Accessibility & Performance

Test keyboard navigation, focus visibility, contrast, labels, reduced motion, image sizes, responsive behavior, loading performance, and layout stability.

5. Security & Deployment

Use HTTPS, avoid exposing privileged secrets, validate data on the server, keep dependencies updated, check all routes/links, configure DNS/hosting, and test the production build.

Final Project Checklist

Plan -> Build -> Test -> Debug -> Optimize -> Commit -> Deploy -> Retest. A strong final project should work on small and large screens, support keyboard users, load efficiently, use valid links/forms, and have a clean Git history.

Next Step

Create a portfolio website containing at least three pages or sections, responsive navigation, a project gallery, an accessible contact form, one JavaScript interaction, and a deployed live version.